

GPU Implementation of KWH Cooling in PKD- GRAV3

BY BIN LIU

Supervisor: Prof. Robert Feldmann

Email: bin.liu@uzh.ch

Abstract

This report presents a GPU implementation of the KWH radiative cooling in PKDGRAV3, based on the existing CPU version. Cooling is coupled to the leapfrog KDK scheme with adaptive rungs, and cooling data are batched across work packets for efficient CUDA execution. Sod tube tests show close agreement between the GPU and CPU versions.

Table of contents

Abstract	1
1 Introduction	1
2 KWH Cooling Model	2
3 Internal Energy Update Scheme	3
4 Implementation	5
5 GPU Scheduling	6
6 Results	7
7 Acknowledgments	9
8 Declaration of the Use Generative AI Usage	9
Bibliography	10

1 Introduction

Matter in the universe behaves like a fluid as it evolves. However, because of limited computational power, it is infeasible to track every constituent particle in hydrodynamic simulations. PKDGRAV3¹ uses SPH to discretize the fluid into a finite number of SPH particles[3]. These particles carry point-particle quantities such as position, velocity, and acceleration, as well as fluid quantities such as temperature and pressure. In this way, the hydrodynamics of the fluid is represented by the dynamics of the SPH particles.

The force exerted on an SPH particle is primarily the sum of the local hydrodynamic force and Newtonian gravity. For example, the density of an SPH particle depends on the spatial distribution of neighboring particles:

$$\rho_I = \sum_{\text{vicinity}} m_J W(|\mathbf{r}_I - \mathbf{r}_J|) \quad (1)$$

1. <https://bitbucket.org/dpotter/pkdgrav3/src/gpu-cooling/>

Here, W is the kernel function, which depends only on the spatial separation. With this approach, the local hydrodynamic force involves only hundreds of particles.

The gravitational component, which is the cornerstone of the calculation, is a typical N -body problem. Even after the SPH discretization, N can still reach trillions² in cosmological simulations. PKDGRAV3 organizes these particles into a tree structure, using the fact that the gravitational effect of a distant cluster can be approximated by that of a point source. The smallest cluster is called a WP³ in this report. It is a group of particles partitioned during the tree walk and passed to `pkdParticleWorkDone`.

These WPs evolve independently, then drift and are regrouped. The hydrodynamic calculation is performed during their independent evolution and can therefore run in parallel. To preserve CPU resources for tree building, modular hydrodynamic processes such as cooling are computed on the GPU.

2 KWH Cooling Model

This model is designed to simplify the calculation of the radiative cooling rate of primordial gas[1]. Its composition, 76% hydrogen and 24% helium, is fixed by Big Bang nucleosynthesis and is therefore hard-coded[4]. Only H, H⁺, He, He⁺, He⁺⁺, and electrons are relevant.

During various structure-formation processes, the gravitational energy of the gas is converted into heat, which would prevent further collapse in the absence of cooling. Because no cold reservoir is available, primordial gas can cool through only a few processes.

The cooling rate is proportional to the electron number density n_e and usually takes the form

$$\Lambda_s = n_e n_s \sigma_s e^{-\frac{\Upsilon_s}{T}}, \quad (2)$$

where Υ_s is a constant that depends on the particle species $s \in \{\text{H}, \text{H}^+, \text{He}, \text{He}^+, \text{He}^{++}\}$. The cross-section $\sigma_s(T)$, which receives contributions from different channels, is a function of the gas temperature T . The only exception is inverse Compton scattering, for which $\Lambda_C \propto n_e T(1+z)^4$, where z is the redshift. The implemented processes are listed in Table 1.

Channel	Species	Process
Collisional	H, He ⁺	$e^- + s \rightarrow e^- + s^*, s^* \rightarrow s + \gamma$
Recombination	H ⁺ , He ⁺ , He ⁺⁺	$e^- + s^+ \rightarrow s + \gamma$
Dielectronic	He ⁺	$e^- + s^+ \rightarrow s^{**}, s^{**} \rightarrow s + \gamma$
Ionization	H, He, He ⁺	$e^- + s \rightarrow 2e^- + s^+$
Bremsstrahlung	H ⁺ , He ⁺ , He ⁺⁺	$e^- + s^+ \rightarrow e^- + s^+ + \gamma$
Compton	γ	$\gamma + e^- \rightarrow \gamma + e^-$

Table 1. The collisional excitation temperature of helium is too high for this process to be included. Three-body and higher-order processes are also neglected.

- In the excitation channel, a ground-state particle is excited by a collision with an electron without capturing it. The excited particle then emits a photon.
- In the recombination channel, an ion captures an electron and emits a photon.
- In the dielectronic channel, a ground-state atom becomes doubly excited by capturing an electron. The doubly excited particle then emits a photon.
- In the ionization channel, a ground-state particle is ionized by a collision with an electron. The kinetic energy is used to overcome the binding energy.
- In the Compton channel, an electron transfers heat to a CMB photon.

The number density of each species is approximated using the ionization-equilibrium composition. This model may additionally include photoheating by an external ultraviolet background parameterized as a function of redshift. The feature can be controlled by setting `cooling_model` in the `.par` file, as listed in Table 2.

². This report is tested on a 294912-particle example.

³. WP refers to the data that `workParticle*` `wp` points at.

cooling_model	KWH cooling	UV heating
0	Off	Off
1	On	Off
2	On	On

Table 2. For non-cosmological simulations such as the tests in this report, UV heating is disabled.

The final assumption is that the gas is always sufficiently optically thin for a photon to decouple from the gas immediately after being emitted. The KWH cooling model can therefore be summarized by a function⁴ of gas density ρ and temperature T

$$\dot{u}^{\text{cool}} = -\Lambda(\rho, T). \quad (3)$$

This function is further encapsulated⁵ to provide its time integral:

$$\Delta u^{\text{cool}}(\rho, T, \Delta t) = -\int_0^{\Delta t} \Lambda(\rho, T(u)) dt. \quad (4)$$

The conversion between temperature T and internal energy u is given by the ideal-gas equation of state. This report uses them interchangeably.

3 Internal Energy Update Scheme

PKDGRAV3 uses a leapfrog integrator[2] to solve its principal gravitational equation:

$$\mathbf{a}_I^G = \sum_{J \neq I} \frac{G m_J}{|\mathbf{r}_I - \mathbf{r}_J|}, \quad (5)$$

The particle position $\mathbf{r}$ and all accelerations, including the gravitational, hydrodynamic, and total accelerations, are evaluated at integer timesteps such as i , $i \pm 1$, and $i \pm 2$, where $i \in \mathbb{N}$. Velocity is evaluated at half-integer timesteps such as $i \pm \frac{1}{2}$ and $i \pm \frac{3}{2}$.

A code cycle consists of one closing kick, one drift, and one opening kick (KDK). A kick updates the velocity using the acceleration, whereas a drift updates the position using the velocity. The time interval can change at integer steps. For example, the $\Delta t_{i+\frac{1}{2}} = t_{i+1} - t_i$ and $\Delta t_{i-\frac{1}{2}} = t_i - t_{i-1}$ can differ. The interval $\Delta t = \frac{\Delta t_{\text{global}}}{2^{R+1}}$ is controlled by the rung R . The available range of R allows the finest timestep to be on the order of seconds.

To fit this scheme, the internal energy u is also evaluated at half-integer steps, whereas the heating rate $\dot{u}$ is evaluated at integer steps, as shown in Fig.1. The term heating rate emphasizes that $\dot{u}$ is the sum of the hydrodynamic rate of change and the KWH cooling rate.

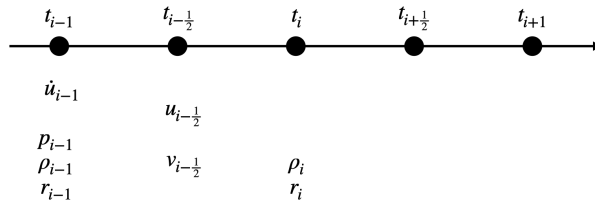


Figure 1. The initial internal energy at timestep t_i is illustrated above. The corresponding density ρ_i is calculated from the particle positions r_i . The internal energy $u_{i-\frac{1}{2}}$ from the previous half-step and the heating rate u_{i-1} from the previous step are recorded for the code step $t_{i-\frac{1}{2}} \rightarrow t_{i+\frac{1}{2}}$.

The internal-energy change $u_i - u_{i-\frac{1}{2}}$ consists of contributions from KWH cooling and hydrodynamics. These two contributions are computed independently and then added together. The KWH contribution Δu_i^{KWH} is calculated using Eq. (4) and the previous internal energy $u_{i-\frac{1}{2}}$.

4. In `pkdgrav3/SPH/sphcooling/SPHCoolingRates.h`

5. `pkdgrav3/SPH/sphcooling/SPHCoolingDeltaU.h` is shared by both CPU and GPU version.

The hydrodynamic contribution is based on the predicted internal energy u_i^{pred} . This prediction assumes that the particle continues from its previous internal energy $u_{i-\frac{1}{2}}$ at the previous heating rate $\dot{u}_{i-1}$. The ideal-gas equation of state then gives the predicted pressure p_i^{pred} from u_i^{pred} . Using this pressure, hydrodynamics provides the rate $\dot{u}_i^{\text{hydro}}$. Multiplying this rate by the time interval gives the hydrodynamic contribution to the internal-energy change, as shown in Fig.2.

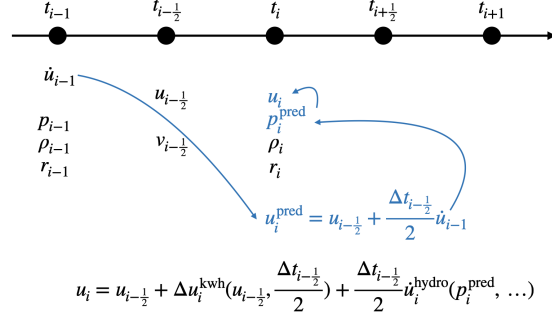


Figure 2. The pressure is used only for hydrodynamics, so it is not updated after u_i is obtained.

As discussed above, the next half-step may have a different time interval; otherwise, the internal-energy change $u_{i+\frac{1}{2}} - u_i$ follows the same pattern. It also consists of contributions from KWH cooling and hydrodynamics. KWH cooling starts from u_i at the same density ρ_i . The hydrodynamic rate $\dot{u}_i^{\text{hydro}}$ also remains the same as in the previous half-step, as shown in Fig.3.

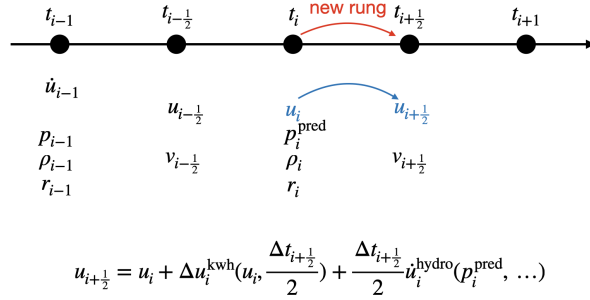


Figure 3. The other half-step uses the same parameters, ρ_i and $\dot{u}_i^{\text{hydro}}$, as the previous half-step.

The overall cooling rate $\dot{u}_i$ for the code step $t_{i-\frac{1}{2}} \rightarrow t_{i+\frac{1}{2}}$ is defined by the internal-energy change and the total time interval, as shown in Fig.4.

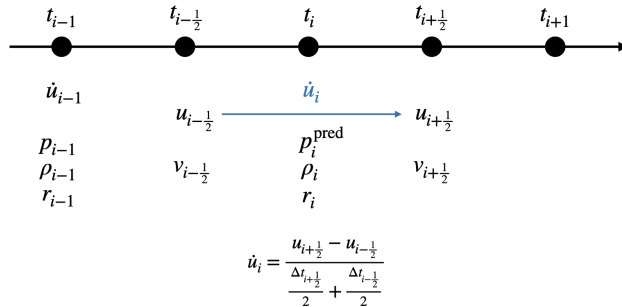


Figure 4. The cooling rate $\dot{u}$ is saved for the next prediction.

A final check is required to ensure that the internal energy predicted from the heating rate $\dot{u}_i$ remains positive at the end of the next code cycle, as shown in Fig.5. A particle undergoing a drastic cooling process may fail to meet this criterion. The timescale of such a process is much shorter than the gravitational dynamical timescale, but the rung for $t_i \rightarrow t_{i+1}$ cannot anticipate this cooling.

When KWH first proposed their model, they imposed a simple limiter under which a particle could lose at most half of its internal energy in one half-step. GPU supercomputers now make it feasible to try smaller timesteps instead. When a particle fails to meet the positivity criterion, the code raises its rung, recalculates the internal energy for $t_i \rightarrow t_{i+\frac{1}{2}}$, and checks the result again. The program aborts if the rung limit is exceeded.

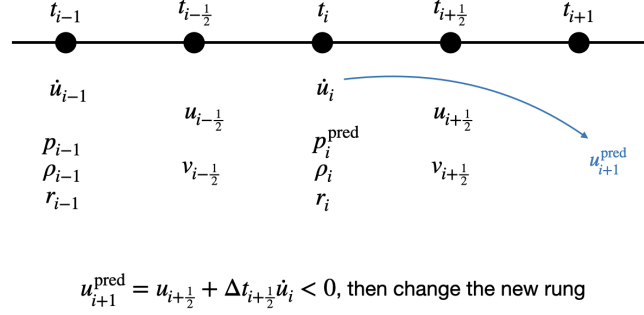


Figure 5. At the end of the code cycle $t_{i-\frac{1}{2}} \rightarrow t_{i+\frac{1}{2}}$, particles are checked to ensure that their internal energy will remain positive at the end of the next code cycle $t_{i+\frac{1}{2}} \rightarrow t_{i+\frac{3}{2}}$.

4 Implementation

The WP is created by `pkdGravInteract`. When it is created, it also queues some GPU tasks.

```
pkdGravInteract{
...
auto wp = new WP;
wp->nRefs=0;
wp->closefinished=0;
queueCooling(wp, bGPU); //bGPU=1 is a flag to turn on GPU.
...
}
```

When a GPU task is submitted, it increments `nRefs`; when it finishes, it decrements `nRefs` and calls `pkdParticleWorkDone`. This system tracks the submitted and completed GPU tasks. The WP is updated in `pkdParticleWorkDone`. Its schematic structure is as follows:

```
pkdParticleWorkDone(WP) {
  if (nRefs==0) {
    if (closefinished==0) {
      ...
      applyCloseKick; //from t_{i-0.5} to t_i
      closefinished=1;
      queueOpenCooling;
      return;
    } else DoOpenKick; //from t_i to t_{i+0.5}
  }
}
```

The main routine in `pkdParticleWorkDone` is triggered immediately after all submitted tasks finish. It has two entry points, depending on the `closefinished` flag. As the name suggests, the default value `closefinished==0` leads to the closing kick $u_{i-\frac{1}{2}} \rightarrow u_i$, whereas `closefinished==1` leads to the opening kick $u_i \rightarrow u_{i+\frac{1}{2}}$. After the closing-kick internal-energy update is complete, the flag is flipped and KWH cooling for the opening kick is submitted. The schematic structure of the positivity check is as follows.

```

DoOpenKick(WP) {
    NeedRecool=0;
    for (every particle inside WP) {
        if (uPred<0) {
            rung++; //This is realized inside function "checkOpenUPredNeedsRecool".
            NeedRecool=1;
        }
        if (NeedRecool==1) {
            queueOpenCooling;
            return;
        }
    }
    applyOpenKick;
}

```

The `NeedRecool` flag indicates if any particle fails the check. Each particle is examined once, and the `rung` of each failed particle is incremented by one. Depending on the flag, the entire WP is either resubmitted for KWH cooling or prepared for the opening-kick update.

5 GPU Scheduling

To improve transfer efficiency, many⁶ WPs copy their cooling data into a shared buffer and send it to the GPU collectively. The same approach is used to return results from the GPU to the CPU, as shown in Fig.6. The tree walk that creates WPs also handles partially filled host buffers at the end⁷.

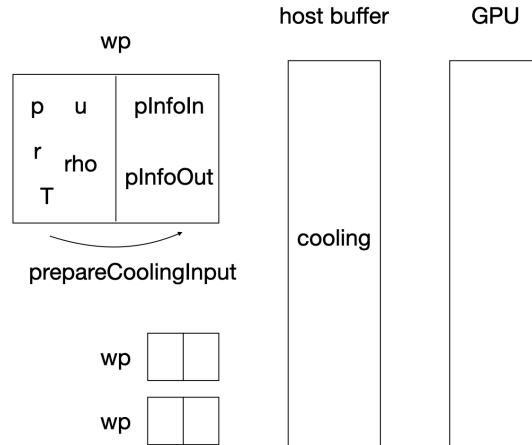


Figure 6. Each WP also has its own `pInfoIn` buffer. The `MessageCooling*` `cooling` class contains the host buffer and kernel function. Particle data are first copied to the WP buffer; the host buffer then gathers data from multiple WP buffers and sends them at once.

⁶ The size of host buffer is defined by `CUDA_WP_MAX_BUFFERED=128` WPs or `requestBufferSize=2MB`.

⁷ `pkdgrav3/gravity/walk2.cxx`

This pattern is shared by close kick cooling and open kick cooling, as shown in Table 3.

	MessageCooling*	submission function	free buffer
close kick	coolingClose	queueCooling	freeCoolingClose
open kick	cooling	queueOpenCooling	freeCooling

Table 3. A host buffer contains both `pHostBufIn` and `pHostBufOut`. Free buffers are stored in a pool in `CudaClient::CudaClient`. A finished task returns its buffer to this pool using `dequeue`, and a submission function can obtain a buffer from the pool using `enqueue`.

For example, the related functions in open kick are

```
queueOpenCooling(WP) {
    prepareCoolingInput(bOpenPass); //copy data to pInfoIn
    pkd->cudaClient->queueOpenCooling;
}
CudaClient::queueOpenCooling(WP) {
    if (cooling) { //a buffer already exists
        if (cooling->queue(WP)) return np; //add WP to the host buffer
        cuda.enqueue(cooling); //can't add WP, host buffer full, submit this buffer
        cooling=nullptr; //delete this host buffer
    }
    gpu.flushCompleted(); //copy the host result back to WP buffer
    cooling=&freeCooling.dequeue; //take a new host buffer
    if (cooling->queue(WP)) return np; //np is the number of particles in this WP
}
MessageCooling::finish() {
    for (auto* wp : ppWP) { //loop over every WP in this host buffer
        for (auto i = 0; i < wp->nP; ++i) { //loop over every particle in this WP
            wp->pInfoOut=Copy2pInfoOut(deltaU_cool,bOpenPass);
        }
    }
    pkdParticleWorkDone(wp);
}
ResetAndFree(this,bOpenPass); // return this host buffer to the free pool.
}
```

Here, the `bOpenPass` flag indicates the source and destination when data are copied. It is `true` because this is the opening kick.

The GPU scheduler automatically finds an available GPU stream and launches the kernel. Since particles cool individually, each particle is assigned to one thread. The grid-block-thread hierarchy is simply one-dimensional, and the key kernel⁸ simply evaluates the explicit integral in Eq. (4).

```
cudaCooling() {
    deltaU_cool[block.Idx][threadIdx.x] = sphCooling_DeltaU_from_rho_T_t;
}
```

6 Results

The same test files⁹ are used to compare the GPU and CPU cooling versions, as shown in Fig.7, Fig.8, Fig.9, and Fig.10. The `analytical` curves are the analytical solutions for the Sod tube without cooling. The `nocool` label denotes `cooling_model=0`, `kwh` denotes `cooling_model=1`, and `kwhheat` and `kwh_heat` denote `cooling_model=2`.

⁸. `pkdgrav3/cuda/cudacooling.cu`

⁹. <https://gitlab.ethz.ch/tozimmermann/bachelor-thesis>

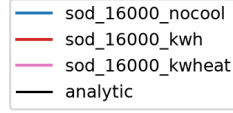


Figure 7. The same color in the following figures always means the same cooling model.

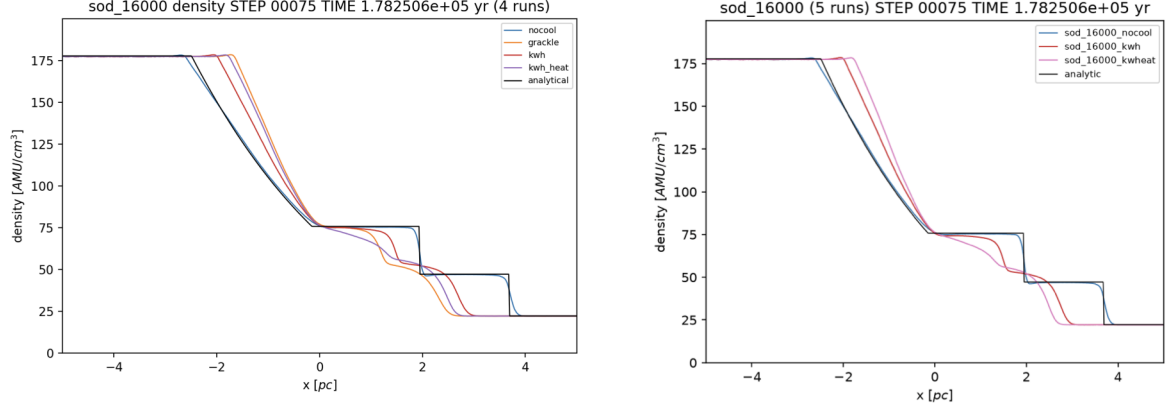


Figure 8. Density comparison between the CPU version (left) and the GPU version (right).

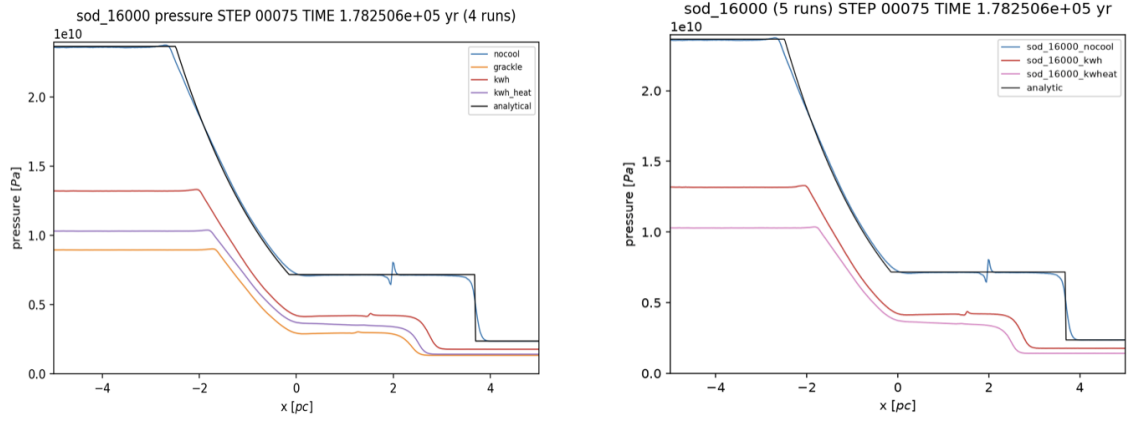


Figure 9. Pressure comparison between the CPU version (left) and the GPU version (right).

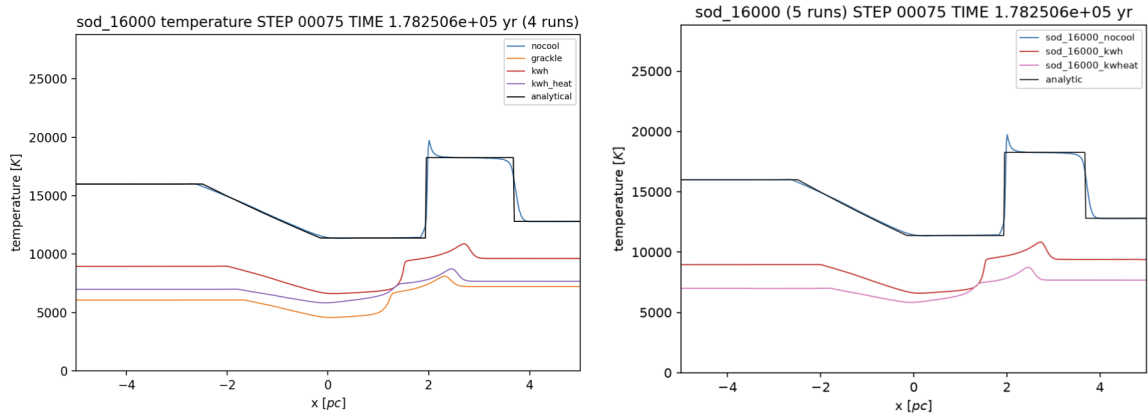


Figure 10. Temperature comparison between the CPU version (left) and the GPU version (right).

Periodic boundary conditions produce two shock surfaces; only half of the simulation box is plotted here. With the left side initially at 16000 K and the right side at 12800 K, the gas does not undergo drastic cooling, so the GPU plots are almost identical to the CPU plots. Because the test case is small, the GPU and CPU versions have similar runtimes with the following configuration¹⁰,

```
#SBATCH --nodes=1
#SBATCH --ntasks-per-node=4
#SBATCH --cpus-per-task=72
#SBATCH --gpus-per-task=1
```

as shown in Table 4. The GPU version of `sod_10_2` takes significantly more time, maybe because it has more various rung hierarchy.

	CPU	GPU
<code>sod_10_1</code>	316	322
<code>sod_10_2</code>	10649	12994
<code>sod_16000_1</code>	394	406
<code>sod_16000_2</code>	514	611
<code>peq_1</code>	653	662
<code>peq_2</code>	1320	1666

Table 4. Here, `sod_10_1` denotes a Sod tube with `cooling_model=1`, with 10 K gas in one half and 8 K gas in the other; `sod_16000_1` denotes a Sod tube with `cooling_model=1`, with 16000 K gas in one half and 12800 K gas in the other; and `peq_1` denotes a tube with `cooling_model=1` and equal pressure on both sides. Runtime is given in seconds.

7 Acknowledgments

I would like to thank Prof. Robert Feldmann for introducing me to the field of astrophysics through his course and for his continued guidance throughout this research project.

I am also grateful to Dr. Douglas Potter for introducing me to high performance computing through his courses and for his help in resolving the various issues I encountered during this project.

My sincere thanks also go to Thomas Meier for his detailed explanations of PKDGRAV3 and his close involvement in our discussions. I wish him every success in his PhD defense.

This work builds upon the CPU implementation developed by Tobias Zimmermann.

Computational resources were provided by the Swiss National Supercomputing Centre on Piz Daint.

8 Declaration of the Use Generative AI Usage

Generative AI tools were used during the preparation of this thesis.

ChatGPT by OpenAI was used for linguistic revision, including spelling and grammar correction and improvements in clarity. Cursor by Anysphere was used in Auto mode to assist with exploring the PKDGRAV3 codebase and drafting GPU-related APIs. In Auto mode, the underlying model was selected by Cursor automatically and was not recorded for each interaction.

All AI-assisted outputs have been reviewed and validated by the author. The author assumes full responsibility for the content of this thesis.

10. <https://github.com/BinLiu33/semester-project-on-gpu-cooling>

Bibliography

- [1] Neal Katz, David H. Weinberg, and Lars Hernquist. Cosmological simulations with TreeSPH. *The Astrophysical Journal Supplement Series*, 105:19-35, jul 1996.
- [2] Thomas Meier, Douglas Potter, Christian Reinhardt, and Joachim Stadel. Smoothed particle hydrodynamics in pkdgrav3 for shock physics simulations. i. hydrodynamics. *The Astrophysical Journal*, 1000(2):266, 2026.
- [3] Douglas Potter, Joachim Stadel, and Romain Teyssier. Pkdgrav3: beyond trillion particle cosmological simulations for the next era of galaxy surveys. *Computational Astrophysics and Cosmology*, 4(1):2, 2017.
- [4] Tobias Ciaran Zimmermann. Introducing primordial radiative cooling to PKDGRAV3. aug 2024. Bachelor's thesis, ETH Zürich and University of Zurich.